
Skill up – Python

저작권이 일부 있으므로 배포를 금지합니다.

Update : 2026-06



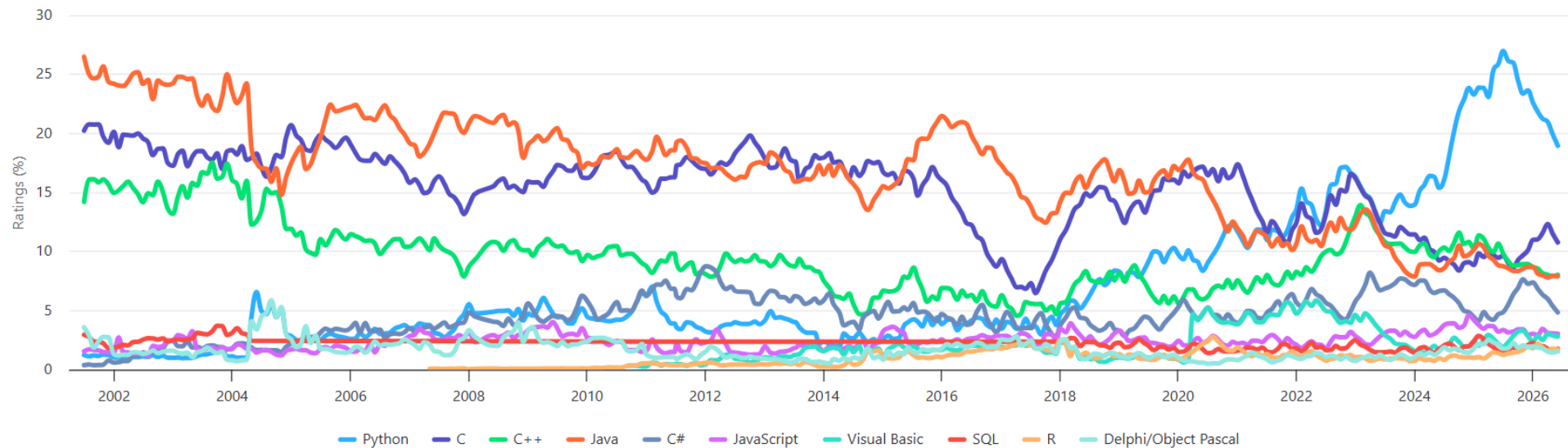
파이썬 이야기

- 1990년 암스테르담의 귀도 반 로섬(Guido Van Rossum)이 크리스마스 기념으로 개발한 인터프리터 언어
- 파이썬 언어 이름의 유래는 창시자가 좋아하는 코미디 쇼인 몬티 파이썬의 날아다니는 서커스(Monty Python's Flying Circus)
- 파이썬의 사전적인 의미는
고대 신화에 나오는
파르나소스 산의 동굴에 살던
큰 뱀



프로그래밍 언어 사용 순위

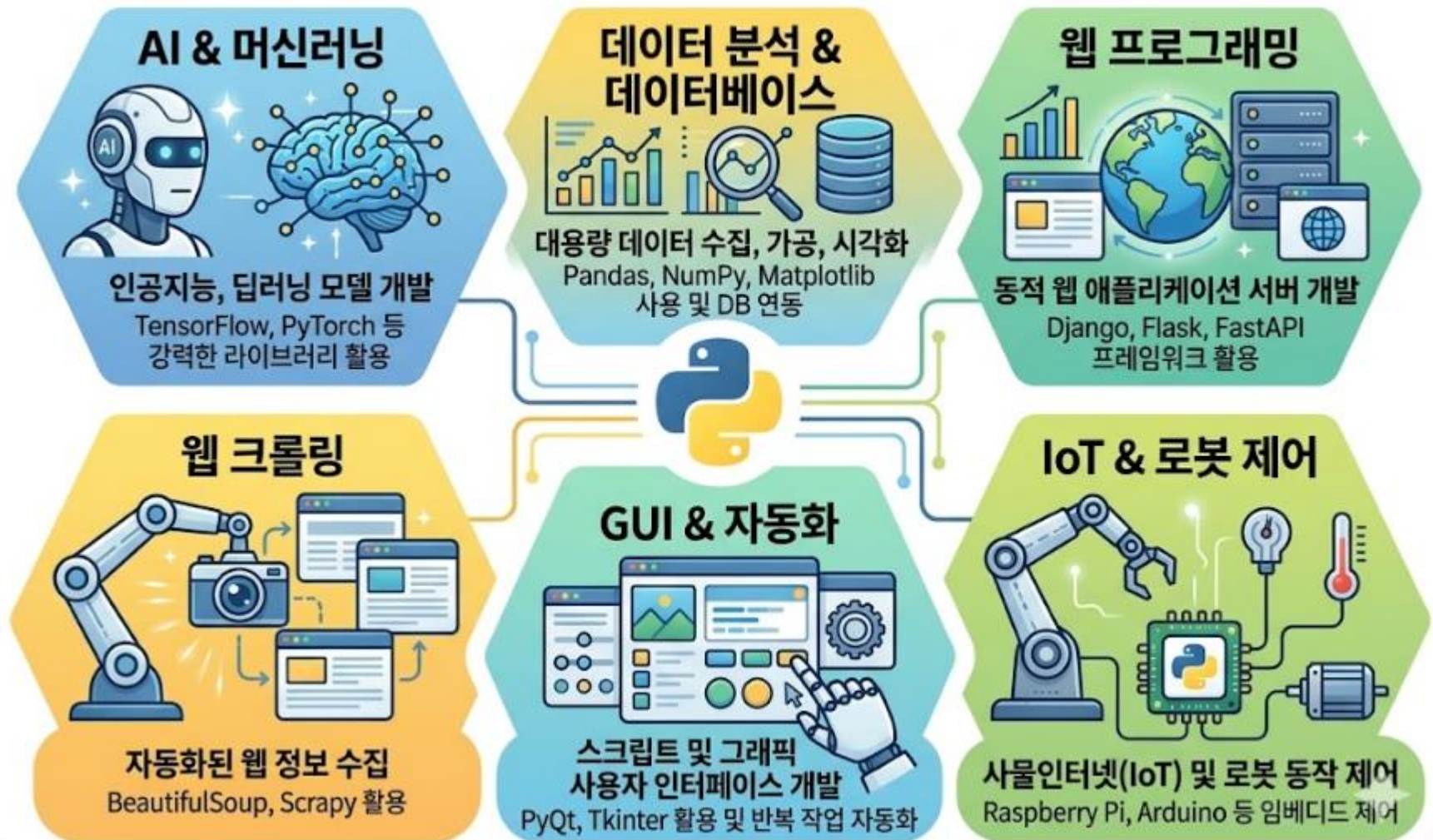
티오베에서 제공 : <https://www.tiobe.com/tiobe-index/>



파이썬의 특징

- 인간 친화적 자연어 스타일
- 동적 타이핑 , 인터프리팅 환경 언어
- AI 및 데이터 과학 생태계를 독점하는 무한한 패키지
- 글로벌 표준 First Language (비전공자 최적화)
- 비용 제로, 구글·MS 인프라도 지탱하는 오픈소스 신뢰성
- 가장 거대하고 활발한 개발자 커뮤니티 보유. 코드 데이터(Stack Overflow, GitHub 등) 대규모 축적

파이썬의 활용분야



주석

- 주석(Comment)란 실행에 상관없는 코드.
- 한줄 주석
 - # comment
 - VS CODE와 같은 텍스트 에디터에서는 [Ctrl]+[/] 단축키 사용가능
- 여러줄 주석 용도로 사용
 - 시작과 종료 부분에 홑따옴표(')를 3개 연속으로 삽입한다.
 - ```
'''
Comment
'''
```

# 출력문

`print(데이터(literal)/계산식/변수)`

```
print("Hello, Python!")
print(2026)
print(3.14)
print(10 + 20)
print("Life is " + "too short")
```



# 이스케이프 문자

이스케이프 문자(Escape character)는 문자열 내에서 특수한 의미를 갖는 문자나 기호를 표현할 때 사용하는 문자로 보통 백슬래시(\)를 사용하여 이스케이프 문자로 표시. 문자열에 포함할 수 없는 특수 문자나 제어 문자를 표현할 때 사용.

`\n`: 줄 바꿈 (new line)

`\t`: 탭 (tab)

`\\`: 백슬래시 (\) 자체

`\'`: 작은따옴표 (single quote)

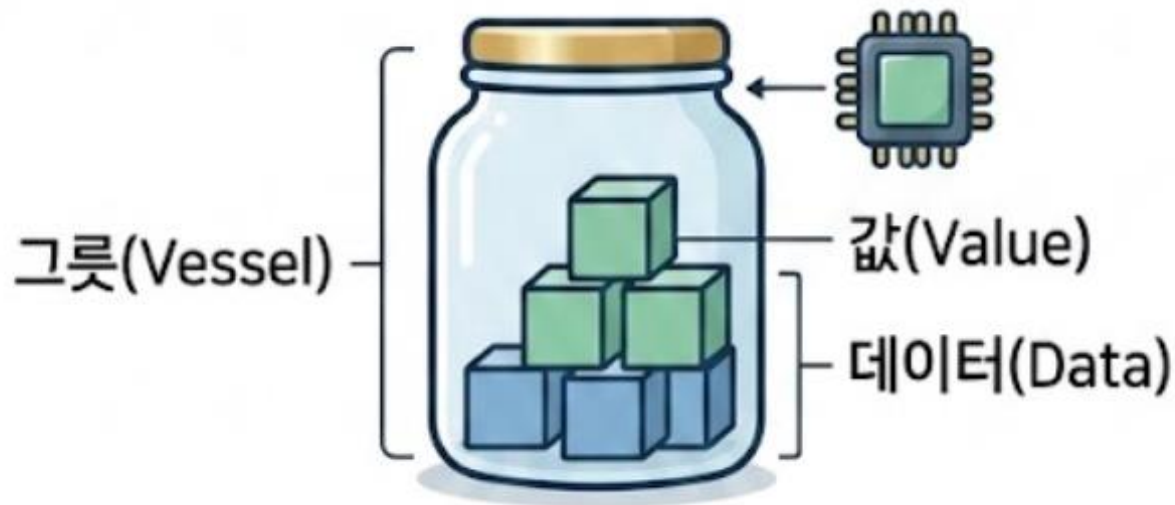
`\"`: 큰따옴표 (double quote)

# 이스케이프 문자

```
\n, \t
txt1 = "김철수"
txt2 = "고길동"
txt3 = "박지민"
print(txt1)
print('\n\t\t',txt2, '\n\n')
print(txt3)
```

# 변수

- 변수란 어떠한 값을 저장하는 메모리 공간(그릇)
- 변수를 만들 때는 대입연산자, =(assignment) 기호를 사용한다.



# 변수

```
3 + 5 + 6 = ?
```

```
계산식 문자열 , 계산식
```

```
print("3 + 5 + 6 =", 3 + 5 + 6)
```

```
변수 이용
```

```
n1 = 3
```

```
n2 = 5
```

```
n3 = 6
```

```
print(n1, "+", n2, "+", n3, "=", n1 + n2 + n3)
```

```
한줄로 변수 선언하기
```

```
a, b, c = 100, 200, 300
```

```
print(a, b, c)
```

# 변수

- 변수명(식별자) 규칙
  - 대 · 소문자를 구분한다( `myVar`와 `MyVar`는 다른 변수).
  - 문자, 숫자, 언더바(`_`)를 포함할 수 있다. 하지만 숫자로 시작하면 안 된다  
( `var2(0)`, `_var(0)`, `var_2(0)`, `2Var(X)`).
  - 예약어는 변수명으로 쓰면 안 된다.  
파이썬의 예약어는 `True`, `False`, `None`, `and`, `or`, `not`, `break`, `continue`, `return`, `if`, `else`, `elif`, `for`, `while`, `except`, `finally`, `global`, `import`, `try` 등이다.
  - 변수명은 의미있는 단어로 표기하는 것이 좋다.

## 변수명 표기법

카멜 표기법 (camel case)



예) `dailyUserTable`

파스칼 표기법 (pascal case)



예) `DailyUserTable`

스네이크 표기법 (snake case)



예) `daily_user_table`

헝가리안 표기법 (Hungarian notation)



예) `strDailyUserTable`



# 변수명 표기법

- 카멜 표기법(camel case)
  - : 단어가 여러개 붙을 때, 앞 단어를 제외한 첫 자를 대문자로 표기
- 파스칼 표기법(pascal case)
  - : 모든 단어의 앞 자가 대문자로 시작
- 스네이크 표기법(snake case)
  - : 각 단어의 구분을 위하여 언더바( \_ )
- 헝가리안 표기법
  - : 접두사에 자료형을 붙임

# 변수

```
예약어는 변수명으로 사용할 수 없다
파이썬의 예약어(keyword) 출력하기
import keyword
print(keyword.kwlist)
print('키워드 갯수 = ', len(keyword.kwlist))
```

# 기본 자료형

- 자료형
  - 프로그래밍을 할 때 쓰이는 숫자, 문자열 등등의 것
  - 숫자형, 문자열, 불린(Boolean)
- 숫자형

| 유형        |     |         | 설명              | 예시              | 선언 형태        |
|-----------|-----|---------|-----------------|-----------------|--------------|
| 수치<br>자료형 | 정수형 | Integer | 양/음의 정수         | 1,2,3,100, -9   | data = 1     |
|           | 실수형 | Float   | 소수점이 포함된 실수     | 10.2, -9.3, 9.0 | data = 9.0   |
| 문자형(문자형)  |     | String  | 따옴표에 들어가 있는 문자형 | abc, a20abc     | data = 'abc' |
| 논리/불린 자료형 |     | Boolean | 참 또는 거짓         | True, False     | data = True  |

# 기본 자료형

- 숫자형
  - 정수
  - 실수

```
n1 = -100 # 정수
n2 = 10.5 # 실수
print(n1, type(n1))
print(n2, type(n2))
```

# 기본 자료형

## ■ 문자열형

- 문자의 나열. 문자들의 집합
- 문자열은 작은 따옴표( ' '), 큰 따옴표( " "), 인용부호 3개로 감싸야 한다.
- 파이썬에서는 JAVA, C언어에 존재하는 문자형(char형)이 따로 없다.

## ■ 문자열 자료형 만드는 4가지 방법

```
"Hello World"
```

```
'Python is fun'
```

```
"""Life is too short, You need python"""
```

```
'''Life is too short, You need python'''
```

# 기본 자료형

- 여러 줄로 이루어진 문자열 - 3개의 인용부호 이용

```
txt1 = "Hello World"
txt2 = 'Python is fun'
txt3 = """Life is too short,
 You need python"""
txt4 = '''
Life is
 too short,
You
 need python
'''
```

```
print(txt1)
print(txt2)
print(txt3)
print(txt4)
```

# 기본 자료형

- 불(Boolean)형
  - 참과 거짓
  - True / 0제외 나머지 숫자 / 글자나 공백등이 삽입된 문자열 / 원소가 삽입된 리스트|튜플|딕셔너리|집합
  - False / 0 / None / 비어있는 문자열 / 비어있는 리스트|튜플|딕셔너리|집합
  - bool(변수|데이터) => 논리형으로 데이터를 변경하는 함수

# 기본 자료형

```
x, y, z = True, False, None
```

```
print(x, type(x))
```

```
print(y, type(y))
```

```
print(z, type(z))
```

```
xx = bool(z)
```

```
print(xx, type(xx))
```

```
수치형데이터 => 논리형으로 변경
```

```
x, y = 0, 100
```

```
xx, yy = bool(x), bool(y)
```

```
print(xx, yy)
```

```
문자열데이터 => 논리형으로 변경
```

```
x, y = 'python', ''
```

```
xx, yy = bool(x), bool(y)
```

```
print(xx, yy)
```



# 자료형 변환 Casting

- 캐스팅(Casting)이란? 데이터간의 형 변환
- 파이썬의 대표적인 캐스팅관련 함수
  - `int()` : 정수형 데이터형으로 변환
  - `float()` : 실수형 데이터형으로 변환
  - `str()` : 문자열 데이터형으로 변환
  - `bool()` : 논리형 데이터형으로 변환
  - `type()` : 데이터 형 반환

```
자동 데이터형 변환
정수 연산자 실수 => 실수
result = n1 + n2
print(result, type(result))
```

# 자료형 변환 Casting

```
문자열형 => 숫자형

n1, n2, n3 = '10', '3.789', '-5'

print(int(n1)) # 10

print(int(n2)) # ValueError

print(int(n3)) # -5

print(float(n1)) # 10.0

print(float(n2)) # 3.789

print(float(n3)) # -5.0
```

# 입력문

- input
  - 사용자가 값을 입력할 때 사용한다.
  - 입력되는 모든 것을 문자열로 처리한다.
- 변수에 저장하기  
변수명 = input("질문")

```
studentName = input('이름을 입력하세요 \n => ')\nstudentAddr = input('주소를 입력하세요 \n => ')\nprint('-'*50)\nprint('학생 이름 => ', studentName,)\nprint('거주지 => ', studentAddr)
```

# 입력문

퀴즈 : 2개의 숫자값을 입력받은 후 사칙 연산을 수행하여라

첫번째 숫자를 입력하세요... 10

두번째 숫자를 입력하세요... 25

-----

$$10 + 25 = 35$$

$$10 - 25 = -15$$

$$10 * 25 = 250$$

$$10 / 25 = 0.4$$

# 입력문

퀴즈 : 학생명과 나이를 입력받아 학생명, 나이, 태어난 년도를 출력하도록  
프로그래밍하여라

학생명... 홍길동

나이...(정수로 입력) 17

-----

학생이름 : 홍길동

나이 : 17 세

태어난 해: 2010 년

# 연산자

## ■ 산술 연산자

| 연산 기호 | 뜻                   | 예시        | 결과                     |
|-------|---------------------|-----------|------------------------|
| +     | 더하기                 | $7+4$     | 11                     |
| -     | 빼기                  | $7-4$     | 3                      |
| *     | 곱하기                 | $7*4$     | 28                     |
| /     | 나누기                 | $7/4$     | 1.75                   |
| **    | 제곱 (같은 수를 여러 번 곱함)  | $2**3$    | 8 (2를 세 번 곱함 $2*2*2$ ) |
| //    | 정수로 나누었을 때의 몫       | $7//4$    | 1 (나눗셈의 몫)             |
| %     | 정수로 나누었을 때의 나머지     | $7\%4$    | 3 (나눗셈의 나머지)           |
| ( )   | 다른 계산보다 괄호 안을 먼저 계산 | $2*(3+4)$ | 14                     |

# 연산자

- 대입 연산자

| 연산자              | 사용 예                 | 설명                           |
|------------------|----------------------|------------------------------|
| <code>+=</code>  | <code>a += 3</code>  | <code>a = a + 3</code> 과 동일  |
| <code>-=</code>  | <code>a -= 3</code>  | <code>a = a - 3</code> 과 동일  |
| <code>*=</code>  | <code>a *= 3</code>  | <code>a = a * 3</code> 과 동일  |
| <code>/=</code>  | <code>a /= 3</code>  | <code>a = a / 3</code> 과 동일  |
| <code>//=</code> | <code>a //= 3</code> | <code>a = a // 3</code> 과 동일 |
| <code>%=</code>  | <code>a %= 3</code>  | <code>a = a % 3</code> 과 동일  |
| <code>**=</code> | <code>a **= 3</code> | <code>a = a ** 3</code> 과 동일 |

# 연산자

```
x = 100
y = 33
print(x, '/', y, '=', x/y)
print(x, '//', y, '=', x//y)
print(x, '%', y, '=', x%y)
print(x, '**', 3, '=', x**3)
print(100-23*2)
print((100-23)*2)
```

```
cnt = 0
print('cnt = ', cnt)
cnt += 10
print('cnt = ', cnt)
cnt -= 3
print('cnt = ', cnt)
cnt *= 2
print('cnt = ', cnt)
```



# 연산자

## ■ 관계 연산자

- 어떤 것이 크거나 작거나 같은지 비교하는 것(참은 True로, 거짓은 False로 표시)
- 반복문, 조건문에 주로 사용

| 연산자 | 의미      | 설명            |
|-----|---------|---------------|
| ==  | 같다.     | 두 값이 동일하면 참   |
| !=  | 같지 않다.  | 두 값이 다르면 참    |
| >   | 크다.     | 왼쪽이 크면 참      |
| <   | 작다.     | 왼쪽이 작으면 참     |
| >=  | 크거나 같다. | 왼쪽이 크거나 같으면 참 |
| <=  | 작거나 같다. | 왼쪽이 작거나 같으면 참 |

```
print(100 == 0)
print(100 != 0)
print(100 > 0)
print(100 < 0)
print(100 >= 100)
```

# 연산자

- 논리 연산자

조건이 참인지 거짓인지 판단할 때 사용하는 연산자  
조건문에 주로 사용

| 연산자       | 의미       | 설명             | 사용 예                     |
|-----------|----------|----------------|--------------------------|
| and(논리곱)  | ~이고, 그리고 | 둘 다 참이어야 참     | (a > 100) and (a < 200)  |
| or(논리합)   | ~이거나, 또는 | 둘 중 하나만 참이어도 참 | (a == 100) or (a == 200) |
| not(논리부정) | ~아니다, 부정 | 참이면 거짓, 거짓이면 참 | not(a < 100)             |

# 연산자

```
user_id = 'abcd'
user_pwd = '1234abcd'
user_age = 22
print((user_id == 'abcd') and (user_pwd == '1234abcd'))
print((user_id == 'abcd') and (user_pwd == '가나다라'))
print((user_id == '도레미파') or (user_pwd == '1234abcd'))
print((user_id == '도레미파') or (user_pwd == '가나다라'))
print(user_age > 19, not(user_age > 19))
```

# 연산자

- 연산자 우선 순위  
: 여러 개의 연산자가 있을  
경우 정해진 순서

| 우선순위 | 연산자                      | 의미                  |
|------|--------------------------|---------------------|
| 1    | () [] {}                 | 괄호, 리스트, 딕셔너리, 세트 등 |
| 2    | **                       | 지수                  |
| 3    | + - ~                    | 단항 연산자              |
| 4    | * / % //                 | 산술 연산자              |
| 5    | + -                      |                     |
| 6    | << >>                    | 비트 시프트 연산자          |
| 7    | &                        | 비트 논리곱              |
| 8    | ^                        | 비트 배타적 논리합          |
| 9    |                          | 비트 논리합              |
| 10   | <> <=                    | 관계 연산자              |
| 11   | == !=                    | 동등 연산자              |
| 12   | = %= /= //= -= += *= **= | 대입 연산자              |
| 13   | not                      | 논리 연산자              |
| 14   | and                      |                     |
| 15   | or                       |                     |
| 16   | if ~ else                | 비교식                 |

# 문자열 연산자

- 문자열 더해서 연결하기 (Concatenation) : + 이용
- 문자열 곱하기 : \* 이용. 문자열의 반복

```
print('='*17)
c1 = 'Python'
c2 = 'HTML/CSS'
c3 = 'SQLITE/Django'
c4 = 'Scrap'
c5 = c1 + ',' + c2 + ',' + c3 + ',' + c4
print(c5)
```

# 문자열 인덱싱

- 인덱싱(Indexing) : 문자열의 위치를 숫자로 표시

인덱싱의 첫 위치는 0

인덱싱의 마지막 위치는 -1

문자열변수[인덱스값]

```
txt1 = "가나다라마바사"
print('txt1 => ', txt1)
print('txt1[0] => ', txt1[0])
print('txt1[2] => ', txt1[2])
print('txt1[5] => ', txt1[5])
print('txt1[-1] => ', txt1[-1])
print('txt1[-3] => ', txt1[-3])
print('txt1[-5] => ', txt1[-5])
```

# 문자열 슬라이싱

- 문자열 변수[start:end:step]
- start ~ (end-1) 인덱스에 해당하는 값 반환
- start 시작위치 , end 끝위치, step 단계
- start 생략시 처음부터, end 생략시 끝까지, step 생략시 1

```
msg = '0123456789'
print('msg => ', msg)
print('msg[0:3] => ', msg[0:3])
print('msg[-5:-1] => ', msg[-5:-1])
print('msg[3:] => ', msg[3:])
print('msg[-3:] => ', msg[-3:])
print('msg[:5] => ', msg[:5])
print('msg[:-5] => ', msg[:-5])
```

```
print(msg, 'msg[::2] => ', msg[::2])
print(msg, 'msg[1::2] => ', msg[1::2])
print(msg, 'msg[1:7:2] => ', msg[1:7:2])

print(msg, 'msg[::-1] => ', msg[::-1])
print(msg, 'msg[::-2] => ', msg[::-2])
```

## 퀴즈

홍길동씨의 주민등록번호는 881120-1068234 이다.

주민등록번호를 변수로 정의한 후 다음과 같이 출력하여라

연월일 : 881120  
숫자 : 1068234  
881120-\*\*\*\*\*



# 문자열 포매팅

- 문자열 포맷(Format) 이란?

문자열 내에 포맷기호({, %})를 이용하여 변수와 값등을 삽입하여 새로운 문자열로 반환하는 기능

%를 이용한 포매팅

format() 함수를 이용한 포매팅

f" 포매팅. 파이썬 3.6 이상에서 사용 가능

# 문자열 포매팅

- %서식 이용

`print( "%d %d" % ( 100 , 200 ) )`

| 서식         | 값의 예                 | 설명                  |
|------------|----------------------|---------------------|
| %d, %x, %o | 10, 100, 1234        | 정수(10진수, 16진수, 8진수) |
| %f         | 0.5, 1.0, 3.14       | 실수(소수점이 붙은 수)       |
| %c         | "b", "한"             | 한 글자                |
| %s         | "안녕", "abcdefg", "a" | 두 글자 이상인 문자열        |

- %서식 이용 : 소숫점 처리

`print( " %전체자리수.소숫점자리f " % (value) )`

`Print( ' %10.5f ' % (124.7874635) )`

`>>> 124.78746`

# 문자열 포매팅

```
number2 = 123.4567899999
```

```
print('실수1 => %f' % number2)
```

```
print('실수2 => %.3f' % number2)
```

```
print('실수3 => %10.3f' % number2)
```

```
print('실수4 => %20.3f' % number2)
```

```
print('실수5 => %e' % number2)
```

# 문자열 포매팅

- format 함수를 이용한 포매팅
  - 인덱스가 생략된 형태 {}
  - 인덱스가 있는 형태의 format 함수 스타일 {인덱스}{인덱스:서식}
  - 변수에 설정값이 있는 형태로 {변수 이용}
  - 소숫점 처리 {인덱스:전체자릿수.소수점이하자릿수f}

```
pi = 3.14156748
print('pi => {}'.format(pi))
print('pi => %.3f' % pi)
print('pi => {:.3f}'.format(pi))
print('pi => {0:.4f}'.format(pi))
print('pi => {0:10.3f}'.format(pi))
print('pi2 => {pi2:.3f}'.format(pi2 = 3.14156748))
```

# 문자열 포매팅

```
today = '화요일'
```

```
yesterday = '월요일'
```

```
' ~ {} {} ~'.format(변수1, 변수2) : 인덱스 생략방식
```

```
print('오늘은 {}, 어제는 {}'.format(today, yesterday))
```

```
' ~ {인덱스1} {인덱스2} ~'.format(변수1, 변수2)
```

```
print('어제는 {1} , 오늘은 {0}'.format(today, yesterday))
```

```
' ~ {변수1} {변수2} ~'.format(변수1=값1, 변수2=값2)
```

```
print('어제는 {y1} , 오늘은 {y2:s}, 내 생일은 {y3:d}월'.format(y1='월요일', y2='화요일', y3=12))
```

# 문자열 포매팅

- f' 문자열 포매팅 : 3.6 이상 지원

f' 문자열 {변수명이나 변수를이용한수식}'

f' 문자열 {변수명:전체자릿수.소숫점이하자릿수f}'

```
today = '화요일'
```

```
yesterday = '월요일'
```

```
print(f'오늘은 {today} , 어제는 {yesterday}')
```

# 문자열 포매팅

- f '포매팅' 소수점 처리

f' 문자열 {변수명:전체자릿수.소숫점이하자릿수f} '

```
pi = 3.14156748
```

```
print(f'pi => {pi}')
```

```
print(f'pi => {pi:.3f}')
```

```
print(f'pi => {pi:20.3f}')
```

# 문자열 포매팅

- f' 문자열 공백, 대체문자여백 지정
  - f' 문자열 {변수명:>숫자} : 왼쪽여백생성
  - f' 문자열 {변수명:<숫자} : 오른쪽여백생성
  - f' 문자열 {변수명:^숫자} : 좌우여백생성
  - f' 문자열 {변수명:대체문자>숫자} : 왼쪽에 대체문자반복
  - f' 문자열 {변수명:대체문자<숫자} : 오른쪽에 대체문자반복
  - f' 문자열 {변수명:대체문자^숫자} : 좌우 대체문자 반복



# 문자열 포매팅

```
message = 'Hello world'
print(f' *** {message} ***')
print(f' *** {message:>30} ***')
print(f' *** {message:<30} ***')
print(f' *** {message:^30} ***')
print(f' *** {message:#^30} ***')
print(f' *** {message:!!>30} ***')
print(f' *** {message:!!<30} ***')
```

```
*** Hello world ***
*** Hello world ***
*** Hello world ***
*** Hello world ***
*** #####Hello world##### ***
*** !!!!!!!!!!!!!!!!!!!!!Hello world ***
*** Hello world!!!!!!!!!!!!!!!!!!!! ***
```

## 퀴즈

홍길동 씨의 과목별 점수를 이용하여 평균 점수를 소숫점 둘째자리까지 출력하여라

|    |   |       |
|----|---|-------|
| 국어 | : | 86    |
| 영어 | : | 77    |
| 수학 | : | 55    |
| 평균 | : | 72.67 |

# 문자열 함수

- 특정 문자열의 갯수 출력  
문자열변수.count(찾고자하는문자열)
- 특정 문자열의 시작 인덱스 위치 반환  
문자열변수.find(찾고자하는문자열)
- 찾고자 하는 문자열이 없다면 -1 반환  
문자열변수.index(찾고자하는문자열)
- 문자열 교체하기  
문자열변수.replace(찾고자하는문자열, 교체문자열)

# 문자열 함수

- 연결문자 지정하기  
'연결문자'.join(문자열변수)
- 대소문자 변환  
문자열변수.upper()  
문자열변수.lower()  
문자열변수.title()
- 공백 제거하기  
문자열변수.strip()  
문자열변수.rstrip()  
문자열변수.lstrip()
- 문자열 나누기  
문자열변수.split(구분문자)
- 문자열 정렬하기  
문자열변수.center(길이)  
문자열변수.ljust(길이)  
문자열변수.rjust(길이)

## 샘플 문자열

```
샘플 문자열 만들기
```

```
http://www.lipsum.com 이용
```

```
sampleTxt = """Lorem Ipsum is simply dummy text of the printing and typesetting industry.
```

```
Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.
```

```
"""
```

```
result1 = sampleTxt.count('Lorem')
result2 = sampleTxt.count('it')
print(f'Lorem 단어의 빈도수는? {result1}')
print(f'it 단어의 빈도수는? {result2}')
```

```
print(f'첫번째 m의 시작위치는? {sampleTxt.find("m")}')
print(f'첫번째 m의 시작위치는? {sampleTxt.index("m")}')
```

```
찾고자 하는 글자가 없다면 -1
print(f'첫번째 가의 시작위치는? {sampleTxt.find('가')}')
아래는 에러 발생 : ValueError
print(f'첫번째 가의 시작위치는? {sampleTxt.index("가")}')
```

```
print(sampleTxt.replace('is', 'was'))
print()
print(f'is 의 빈도수는? {sampleTxt.count("is")}')
```

```
교체된 내용을 원본 변수에 반영
sampleTxt = sampleTxt.replace('is', 'was')
print(sampleTxt)
print(f'is 의 빈도수는? {sampleTxt.count("is")}')
```

```
cls1 = 'Python'
cls2 = 'Django'
print('=' .join(cls1))
print('...' .join(cls2))
```

```
원본 변수 데이터는 그대로임
print(cls1, cls2)
```

```
가공한 결과를 원본 데이터에 적용
cls2 = '...' .join(cls2)
print(cls1, cls2)
```

```
중간 여백은 삭제 X, 좌우 여백만 삭제
word = ' P y t h o n '
print(f'**{word}**')
print(f'**{word.strip()}**')
print(f'**{word.rstrip()}**')
print(f'**{word.lstrip()}**')
```

```
모든 여백 삭제는 replace() 활용
print(f'**{word.replace(' ', '')}**')
```



```
sample = "동해물과 백두산이 마르고 닳도록"
```

```
print('***',sample.center(50),'***')
```

```
print('***',sample.ljust(50),'***')
```

```
print('***',sample.rjust(50),'***')
```

```
*** 동해물과 백두산이 마르고 닳도록 ***
*** 동해물과 백두산이 마르고 닳도록 ***
*** 동해물과 백두산이 마르고 닳도록 ***
```

## 퀴즈

입력한 이메일 주소 데이터를 정제된 형태로 변경하여 출력하도록 프로그래밍 하여라

정제할 이메일 주소를 입력하세요: `Dream_AI@Naver.Com`

-----

원본 데이터 : [ `Dream_AI@Naver.Com` ]

-----

1. 공백 제거 및 소문자 변환 결과 : `dream_ai@naver.com`
2. 이메일 내 '@' 기호의 개수 : 1 개
3. 이메일 내 '@' 기호의 시작 위치 : 8 번 인덱스

# 퀴즈

(1절)

동해물과 백두산이 마르고 닳도록  
하느님이 보우하사 우리나라만세

(후렴) 무궁화 삼천리 화려강산  
대한사람 대한으로 길이 보전하세

(2절)

남산위에 저 소나무 철갑을 두른듯  
바람서리 불변함은 우리기상 일세

(후렴) 무궁화 삼천리 화려강산 대한사람 대한으로 길이보전하세

## 퀴즈

1) 애국가 1,2 절에서 무궁화는 몇 번 나올까?

예시)

무궁화는 애국가 1~2절에서 ?번 나옵니다

2) 애국가 1,2 절에서 '소나무'의 위치 인덱스 값은?

예시)

애국가 1~2절에서 "소나무"의 인덱스 위치는 ?입니다

3) 애국가 1,2 절에서 줄바꿈 없이 출력하여라.

## 퀴즈

4) 애국가 1절에서 아래 문자열만 출력하여라(슬라이싱, find() 활용)

예시)

동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리나라 만세

5) 애국가에서 일부 슬라이싱 한 후 아래와 같이 출력하여라 (join() 활용)

예시)

동=해=물=과= =백=두=산=이= =마=르=고=

# 리스트 자료형

## 특징

다수개의 데이터 값을 함께 저장하고 호출할 수 있는 컨테이너.

다른 언어와는 다르게 서로 다른 데이터 형을 함께 사용할 수 있다

생성 : 빈 리스트 생성 후 데이터 추가 방식, 초기값 설정 방식

리스트명 = []

리스트명 = [값1, 값2...]

# 리스트 자료형

| 함수        | 설명                                             | 사용법                 |
|-----------|------------------------------------------------|---------------------|
| append()  | 리스트 맨 뒤에 항목을 추가한다.                             | 리스트명.append(값)      |
| pop()     | 리스트 맨 뒤의 항목을 빼낸다(리스트에서 해당 항목이 삭제된다).           | 리스트명.pop()          |
| sort()    | 리스트의 항목을 정렬한다.                                 | 리스트명.sort()         |
| reverse() | 리스트 항목의 순서를 역순으로 만든다.                          | 리스트명.reverse()      |
| index()   | 지정한 값을 찾아 해당 위치를 반환한다.                         | 리스트명.index(찾을값)     |
| insert()  | 지정된 위치에 값을 삽입한다.                               | 리스트명.insert(위치, 값)  |
| remove()  | 리스트에서 지정한 값을 삭제한다. 단 지정한 값이 여러 개면 첫 번째 값만 지운다. | 리스트명.remove(지울값)    |
| extend()  | 리스트 뒤에 리스트를 추가한다. 리스트의 더하기(+) 연산과 기능이 동일하다.    | 리스트명.extend(추가할리스트) |
| count()   | 리스트에서 해당 값의 개수를 센다.                            | 리스트명.count(찾을값)     |
| clear()   | 리스트의 내용을 모두 지운다.                               | 리스트명.clear()        |
| del()     | 리스트에서 해당 위치의 항목을 삭제한다.                         | del(리스트명[위치])       |
| len()     | 리스트에 포함된 전체 항목의 개수를 센다.                        | len(리스트명)           |
| copy()    | 리스트의 내용을 새로운 리스트에 복사한다.                        | 새리스트=리스트명.copy()    |
| sorted()  | 리스트의 항목을 정렬해서 새로운 리스트에 대입한다.                   | 새리스트=sorted(리스트)    |

## 리스트 자료형

```
myList1 = [10, 3.14, 'Hello python', True, False]
```

```
print(f'myList1 => {myList1}, 길이 => {len(myList1)}')
```

```
myList2 = []
```

```
print(f'myList2 => {myList2}, 길이 => {len(myList2)}')
```

```
myList2.append(100)
```

```
myList2.append(200)
```

```
print(f'myList2 => {myList2}, 길이 => {len(myList2)}')
```

```
myList2.insert(0, 10000)
```

```
myList2.insert(1, 20000)
```

```
print(f'myList2 => {myList2}, 길이 => {len(myList2)}')
```



## 리스트 자료형

리스트이름[숫자]

0부터 시작, 인덱스값이 음수(-)인 경우에는 뒤에서부터 순서 지정

리스트이름[start:end:step]

리스트이름[start:end]

리스트이름[start:]

리스트이름[:end]

리스트이름[::step]

리스트이름[:] = 리스트이름[::-1]

```
myList1 = [10, 3.14, 'Hello python', True, False]
```

```
print(myList1[0], myList1[-1])
```

```
print(myList1[1:3])
```

```
print(myList1[::2])
```

```
print(myList1[::-1])
```

## 리스트 자료형

리스트 데이터 갱신 (Update)

- 리스트명[인덱스] = 데이터
- 리스트명[start:end] = 데이터

```
myList3 = ['짜장', '육계장', '로제 스파게티', '탕수육']
print(f'myList3 => {myList3}')
myList3[0] = '짬뽕'
print(f'myList3 => {myList3}')
myList3[2:] = ['크림 파스타', '라면']
print(f'myList3 => {myList3}')
myList3[2:] = '떡국'
print(f'myList3 => {myList3}')
myList3 = ['단팥빵']*4
print(f'myList3 => {myList3}')
```

## 리스트 자료형

리스트 연결 : +  
리스트 반복 : \*

```
city_list1 = ['서울', '부산', '대구']
city_list2 = ['전주', '청주']
print(city_list1 + city_list2)
print(city_list2*3)
```

## 리스트 자료형

리스트명.remove(데이터) : 데이터값을 이용한 삭제

리스트명.pop() : 마지막 데이터 값 삭제, 해당값 반환

리스트명.pop(인덱스) : 인덱스 위치에 해당하는 값이 삭제, 해당값 반환

리스트명.clear() : 데이터 모두 삭제. 빈리스트로 변경

del 리스트명 : 리스트 변수 삭제

```
city_list = ['제주', '서울', '부산', '대구', '대전', '공주', '마산', '세종']
print(city_list)
city_list.remove('대구')
print(city_list)
```

## 리스트 자료형

```
print(f'삭제 값은? {city_list.pop()}')
```

```
print(city_list)
```

```
print(f'삭제 값은? {city_list.pop(0)}')
```

```
print(city_list)
```

```
city_list.clear()
```

```
print(city_list)
```

```
del city_list
```

```
print(city_list) # 오류 발생
```

## 리스트 자료형

# 리스트정렬

리스트명.sort()

sorted(리스트명), sorted(리스트명, reverse=True)

```
city_list = ['제주', '서울', '부산', '대구', '대전', '공주', '마산', '세종']
```

```
print(city_list)
```

```
city_list.sort()
```

```
print(city_list)
```

```
city_list.reverse()
```

```
print(city_list)
```

## 리스트 자료형

```
num_list = [100, 20, -90, 10000, 4, 67]
print(num_list)
print(sorted(num_list)) # 오름차순
print(sorted(num_list, reverse=True)) # 내림차순
print(num_list)

정렬 결과를 다시 원본 리스트에 반영
num_list = sorted(num_list)
print(num_list)
```

# 퀴즈

데이터를 입력받은 후 리스트에 추가하고 결과를 출력하여라.  
( input() 이용, 리스트의 원소는 5개로 한다. )

좋아하는 음식은? 초밥

최근 본 영화는? 알라딘

좋아하는 가수는? BTS

좋아하는 숫자? 10

최근 여행지? 부산

당신에 관한 리스트 : ['초밥', '알라딘', 'BTS', 10, '부산' ]



# 퀴즈

아래와 같이 리스트를 정의하고 다음과 같이 출력하여라.

```
foods = ['사과','망고','치즈케이크','주스']
```

우리집 냉장고에는? ['사과', '망고', '치즈케이크', '주스']

동생이 사과 을/를 먹었다

우리집 냉장고에는? ['망고', '치즈케이크', '주스']

이모가 수박 을/를 사오셨다.

우리집 냉장고에는? ['망고', '치즈케이크', '주스', '수박']

동생 친구가 치즈케이크 , 수박 을/를 먹었다.

우리집 냉장고에는? ['망고', '주스']

## 퀴즈

태어난 년도를 입력받은 후 태어난 년도에 대한 띠를 출력한다

태어난 년도를 12로 나눈후 나머지를 구한다. 나머지에 따라서 띠를 출력한다.

띠 = 태어난년도%12

원숭이, 닭, 개, 돼지, 쥐, 소, 범, 토끼, 용, 뱀, 말, 양 (0 ..... 11)

출력 예시)

태어난 년도를 입력하세요? ... 2009

2009 년도 출생은 소띠입니다.

## 리스트 자료형

# 리스트안에 리스트

리스트안에 삽입되는 데이터는 리스트의 길이가 달라도 되며 리스트가 아닌 데이터가 올 수도 있다

중첩 리스트 인덱싱 : 리스트명[인덱스1][인덱스2]

```
list2 = [[10, 20, 30],
 [100, 200, 300],
 [1000, 2000, 3000]]

print(list2)
print(f'1행1열의 값은? {list2[0][0]}')
print(f'3행3열의 값은? {list2[2][2]}')
print(f'3행3열의 값은? {list2[-1][-1]}')
```

# 리스트 자료형

```
리스트안의 데이터 형태가 다른 형태
list3 = ['고길동',
 [100, 200, 300],
 [1000, 2000, 3000, 4000],
 True]
print(list3, len(list3))
print(list3[0])
print(list3[1])
print(list3[1][2])
print(list3[2][2])
```

## 리스트 자료형

# 기존 리스트를 이용한 중첩 리스트 생성

```
kor = [100, 80, 85]
```

```
math = [55, 70, 35]
```

```
eng = [80, 80, 100]
```

```
python = [90, 70, 88]
```

# 1차원 리스트의 조합 => 중첩리스트 생성

```
grade = [kor, math, eng, python]
```

```
print(f' grade 리스트는? {grade}')
```

```
python_tot = grade[3][0] + grade[3][1] + grade[3][2]
```

```
print(f' python 과목 = {grade[3][0]}, {grade[3][1]}, {grade[3][2]}')
```

```
print(f' python 과목 총점 = {python_tot}')
```

## 리스트 자료형

```
kor_tot = grade[0][0] + grade[0][1] + grade[0][2]
```

```
math_tot = grade[1][0] + grade[1][1] + grade[1][2]
```

```
eng_tot = grade[2][0] + grade[2][1] + grade[2][2]
```

```
python_tot = grade[3][0] + grade[3][1] + grade[3][2]
```

```
print(f' kor : 합계 = {kor_tot} , 평균 = {(kor_tot/3):.2f}')
```

```
print(f' math : 합계 = {math_tot} , 평균 = {(math_tot/3):.2f}')
```

```
print(f' eng : 합계 = {eng_tot} , 평균 = {(eng_tot/3):.2f}')
```

```
print(f' python : 합계 = {python_tot} , 평균 = {(python_tot/3):.2f}')
```

```
kor : 합계 = 265 , 평균 = 88.33
```

```
math : 합계 = 160 , 평균 = 53.33
```

```
eng : 합계 = 260 , 평균 = 86.67
```

```
python : 합계 = 248 , 평균 = 82.67
```

## 퀴즈

매장에 남아있는 음료수들의 재고를 파악하고,  
각 음료수별 총 자산 가치(재고 수량 \* 단가)를 자동으로  
계산해 주는 프로그램을 작성하여라.

- 데이터는 아래 구조의 중첩 리스트(리스트 안의 리스트)로 정의 한 후 사용한다.
- 문자열 포매팅(%, format(), f") 등을 이용하여 출력한다.

```
drink_inventory = [
 ["콜라", 5, 1500],
 ["사이다", 3, 1200],
 ["보리차", 10, 2000],
 ["삼다수", 30, 800],
 ["캔커피", 7, 2500],
]
```

# 퀴즈

=== 편의점 음료수 재고 관리 프로그램 ===

-----  
[상품 정보 확인]

- 1번 상품명 : 콜라, 1500원, 재고 5
- 4번 상품명 : 삼다수, 800원, 재고 30

-----  
[재고 가치 평가 결과]

- 콜라 재고의 총 가치 : 7500 원
- 삼다수 재고의 총 가치 : 24000 원



# 튜플 자료형

- 튜플의 특징

- 리스트는 대괄호 []로 생성, 튜플은 소괄호 ()로 생성
- 튜플은 값을 수정할 수 없으며, 읽기만 가능해 읽기 전용 자료를 저장할 때 사용

- 튜플 선언

- 여러 개 있는 경우 `tupleName = (item1, ... )`
- 한 개만 있는 경우 쉼표(,)로 마무리 `tupleName = (item,)`
- 빈값이 있는 경우 `tupleName = ()`

## 튜플 자료형

```
t1 = (100, 200, 'Python', 3.14, True)
```

```
2개이상일때 () 를 생략 가능
```

```
t2 = '홍길동', '김남길', 100
```

```
빈 튜플
```

```
t3 = ()
```

```
print(t1, type(t1))
```

```
print(t2, type(t2))
```

```
print(t3, type(t3))
```

```
t4 = (11) # 수치형 데이터로 인식
```

```
print(t4, type(t4))
```

```
t5 = ('블랙핑크',) # 튜플 데이터로 인식
```

```
print(t5, type(t5))
```

## 튜플 자료형

- 인덱싱과 슬라이싱

```
yourTuple = ('초밥', '알라딘', 'BTS', 10, '부산')
print(f'yourTuple = {yourTuple}')
print(f'yourTuple[0] = {yourTuple[0]}')
print(f'yourTuple[-1] = {yourTuple[-1]}')
print(f'yourTuple[1:3] = {yourTuple[1:3]}')
print(f'yourTuple[2:] = {yourTuple[2:]}')
print(f'yourTuple[:2] = {yourTuple[:2]}')
print(f'yourTuple[::2] = {yourTuple[::2]}')
print(f'yourTuple[::-1] = {yourTuple[::-1]}')
```

## 튜플 자료형

- 튜플 연산

```
my_tuple = (1, 2, 3)
print(my_tuple*3)
print(my_tuple + my_tuple)
```

- 튜플 데이터 추가

```
cityTuple = ('한국',)
print(f'cityTuple = {cityTuple}')
한개의 튜플 데이터 추가
cityTuple += ('독일',)
print(f'cityTuple = {cityTuple}')
두개의 튜플 데이터 추가
cityTuple += ('벨기에','스위스')
print(f'cityTuple = {cityTuple}')
```

## 튜플 자료형

- 튜플 변수 정의

```
student = (name, age, gender) = ('제인', 27, '여')
```

```
print(f'학생 정보 = {student}')
```

```
print(f'이름 = {name}')
```

```
print(f'나이 = {age}')
```

```
print(f'성별 = {gender}')
```

- 중첩 튜플과 리스트튜플

```
t_2d = ((10, 30), (50, 100), (45, 34))
```

```
tlist_2d = [(10, 30), (50, 100), (45, 34)]
```

```
print(t_2d, type(t_2d), len(t_2d))
```

```
print(tlist_2d, type(tlist_2d), len(tlist_2d))
```

```
print(t_2d[0], t_2d[1][1])
```

```
print(tlist_2d[0], tlist_2d[-1][-1])
```

## 튜플 자료형

- 문자열 => 튜플 : `tuple(문자열변수나 값)`
- 리스트 => 튜플 : `tuple(리스트변수나 값)`
- 튜플 => 리스트 : `list(튜플변수나 값)`
- 튜플 => 문자열

`str(튜플변수나 값)`

구분자.join(튜플변수나 값)

주의사항은 join() 사용시에는 튜플의 자료형이 문자열이어야 한다.

## 튜플 자료형

```
my_text = '가나다라마바사'
```

```
my_tuple = ('초밥', '알라딘', 'BTS', '해운대', '부산')
```

```
my_list = ['파이썬', '자바', 'DB']
```

```
문자열 => 튜플
```

```
temp = tuple(my_text)
```

```
print(my_text , type(my_text))
```

```
print(temp, type(temp))
```

```
리스트 => 튜플
```

```
temp = tuple(my_list)
```

```
print(my_list , type(my_list))
```

```
print(temp, type(temp))
```

## 튜플 자료형

# str() 함수를 이용한 문자열 데이터로 변환

```
temp = str(my_tuple)
```

```
print(my_tuple , type(my_tuple))
```

```
print(temp, type(temp))
```

```
print(temp[0], temp[-3])
```

```
print(len(temp))
```

# join() 함수를 이용한 문자열 데이터로 변환

```
my_tuple = ('초밥', '알라딘', 'BTS', '해운대', '부산')
```

```
temp = ' '.join(my_tuple)
```

```
print(my_tuple , type(my_tuple))
```

```
print(temp, type(temp))
```

```
print(temp[0], temp[-1])
```

```
print(len(temp))
```



## 퀴즈

다음 튜플 데이터의 첫번째에 'fun-coding0' 데이터를 추가하여라

```
tupledata = ('fun-coding1', 'fun-coding2', 'fun-coding3')
```

Before : `tupledata = ('fun-coding1', 'fun-coding2', 'fun-coding3')`

After : `tupledata = ('fun-coding0', 'fun-coding1', 'fun-coding2', 'fun-coding3')`

## 퀴즈

문자열로 구성된 데이터가 초기형태로 삽입된 리스트와 빈 튜플을 선언한 후  
주석문 형태를 참조하여 리스트의 요소를 이용하여 튜플에 요소를 추가하는  
코드를 작성하여라

# 출력 예시

```
step0 : user_list = ['sky_blue', 'admin2026', 'p@ssword', 'membership']
```

```
step1 : user_tuple = ()
```

```
step2 : user_tuple = ('sky',)
```

```
step3 : user_tuple = ('sky', '2026')
```

```
step4 : user_tuple = ('sky', '2026', '@swr')
```

```
step5 : user_tuple = ('membership', 'sky', '2026', '@swr')
```

## 퀴즈

# 리스트 정의 (원본 회원 가입 정보)

```
user_list = ['sky_blue', 'admin2026', 'p@ssword', 'membership']
```

```
print(f' step0 : user_list = {user_list} ')
```

# 비어있는 튜플 정의 (정제된 아이디를 담을 공간)

```
user_tuple = ()
```

```
print(f' step1 : user_tuple = {user_tuple} ')
```

# 퀴즈

- # 1. 리스트의 1번째 요소에서 앞 3글자만 추출하여 튜플에 삽입
- # 2. 리스트의 2번째 요소에서 뒤쪽 숫자 4글자만 추출하여 튜플에 삽입
- # 3. 리스트의 3번째 요소에서 짝수번째 글자만 추출하여 튜플에 삽입
- # 4. 리스트의 4번째 요소 전체를 튜플의 첫번째(맨 앞) 요소로 삽입

# 딕셔너리 자료형

- 연관 배열(Associative array) 또는 해시(Hash)
- Key를 통해 Value를 얻는다
  - 중괄호 {}로 묶어 구성, 키(Key)와 값(Value)의 쌍으로 구성
  - 딕셔너리변수 = {키1:값1, 키2:값2,...}
- 주의할 점
  - : 키 값이 같은 경우 마지막 아이템만 유지된다.
  - : 중복키를 허용하지 않는다
  - : 숫자를 이용한 인덱싱과 슬라이싱 불가능. 키인덱싱만 가능

# 딕셔너리 자료형

- 딕셔너리의 생성

# 문자열키

```
dict1 = {'a':'apart', 'b':'banana', 'c':'cup'}
```

# 숫자키

```
dict2 = {100:'백', 200:'이백', 300:'삼백'}
```

```
print(f'dict1 = {dict1}, 데이터형 => {type(dict1)}')
```

```
print(f'dict2 = {dict2}, 데이터형 => {type(dict1)}')
```

# 딕셔너리 자료형

# 빈딕셔너리 생성

```
dict3 = {}
```

```
print(f'dict3 = {dict3}')
```

# 데이터 추가와 수정 : 딕셔너리명[키] = 값

```
dict3['가'] = '가로수' # 추가
```

```
dict3['장'] = '장미' # 추가
```

```
print(f'dict3 = {dict3}')
```

```
dict3['가'] = '가지' # 수정
```

```
print(f'dict3 = {dict3}')
```

# 딕셔너리 자료형

- 딕셔너리 인덱싱

```
dict1 = {'a':'apart', 'b':'banana', 'c':'cup'}
```

```
dict2 = {100:'백', 200:'이백', 300:'삼백'}
```

```
print(dict1['a'])
```

```
print(dict2[100])
```

# 위치 인덱싱 불가 - 에러발생

```
print(dict1[0])
```

# 슬라이싱 불가

```
print(f'dict3[0:2] = {dict3[0:2]}')
```



# 딕셔너리 자료형

- 딕셔너리 값 변경

```
dict1 = {'a':'apart', 'b':'banana', 'c':'coffee'}
```

```
print(f'dict1 = {dict1}')
```

```
dict1['a'] = 'apple'
```

```
print(f'dict1 = {dict1}')
```

```
update() 함수를 이용한 데이터값 수정
```

```
dict1 = {'a':'apart', 'b':'banana', 'c':'coffee'}
```

```
print(f'dict1 = {dict1}')
```

```
dict1.update(b='baseball') # 수정
```

```
dict1.update(e='egg') # 추가
```

```
print(f'dict1 = {dict1}')
```

```
dict1.update(b='bad', e='elephant') # 수정
```

```
print(f'dict1 = {dict1}')
```

# 딕셔너리 자료형

- 딕셔너리 요소 삭제

```
dict1 = {'a':'apart', 'b':'banana', 'c':'coffee', 'd':'drama', 'e':'egg'}
print(f'dict1 = {dict1}')
dict1.pop('b')
print(f'dict1 = {dict1}')
del dict1['d']
print(f'dict1 = {dict1}')
dict1.clear()
print(f'dict1 = {dict1}')
del dict1
print(f'dict1 = {dict1}') # NameError: name 'dict1' is not defined
```

# 딕셔너리 자료형

- 딕셔너리 함수

values() => 값만 추출, dict\_values 데이터형

keys() => 키만 추출, dict\_keys 데이터형

items() => 리스트안에 (키,값) 튜플 형태로 변환, dict\_items 데이터형

```
sports_dict = {"축구":"손흥민", "피겨":"김연아", "농구":"허웅"}
print(f'sports_dict = {sports_dict}')
print(f'sports_dict.values() = {sports_dict.values()}')
print(f'sports_dict.keys() = {sports_dict.keys()}')
print(f'sports_dict.items() = {sports_dict.items()}')
```

```
리스트 형태로 완전하게 변경하려면?
print(f' 값 리스트 = {list(sports_dict.values())}')
print(f' 키 리스트 = {list(sports_dict.keys())}')
print(f' 리스트안의 튜플 형태 = {list(sports_dict.items())}')
```

# 딕셔너리 자료형

- 딕셔너리 중첩 구조

```
dictList = [{'a':'apple', 'v':'victory'},
 {100:'백', 200:'이백'},
 {'user1':'김철수', 'user2':'안소희'}]
```

```
print(dictList)
print(type(dictList))
print(dictList[0])
print(dictList[2]['user1'])
```

# 딕셔너리 자료형

```
딕셔너리 안에 딕셔너리
```

```
idol_dict = { 'group_name' : '아이브',
 'company' : '스타쉽엔터테인먼트',
 'member' : {
 'vocal' : ['안유진', '장원영', '리즈', '이서'],
 'rapper' : ['가을', '레이'],
 'leader' : '안유진'
 }
}

print(idol_dict)
print(type(idol_dict))

print(idol_dict['group_name'])
print(idol_dict['member'])
print(idol_dict['member']['leader'])
print(idol_dict['member']['vocal'][1])
```

# 딕셔너리 자료형

- 딕셔너리 캐스팅

딕셔너리 값 => 리스트 `list(딕셔너리명.values())`

딕셔너리 키 => 리스트 `list(딕셔너리명.keys())`

딕셔너리 값 => 튜플 `tuple(딕셔너리명.values())`

딕셔너리 키 => 튜플 `tuple(딕셔너리명.keys())`

리스트, 튜플 => 딕셔너리

리스트의 위치인덱스는 키로 지정

`dict(enumerate(리스트|튜플))`

## 딕셔너리 자료형

```
dict1 = {'a':'apart', 'b':'banana', 'c':'cup', 'e':'eat'}
print(list(dict1.values()), tuple(dict1.values()))
print(list(dict1.keys()), tuple(dict1.keys()))
```

```
myList = ['사과', '바나나', '딸기']
temp = enumerate(myList)
print(temp, type(temp))
result_dict = dict(temp)
print(result_dict, type(result_dict))
result_dict2 = dict(enumerate(myList))
print(result_dict2, type(result_dict))
```

## 퀴즈

1) 다음 항목을 딕셔너리(dict)으로 선언하여라.

<성인 - 100000 , 청소년 - 70000 , 아동 - 30000>

2) 1번 에서 선언한 dict 항목에 <소아 - 0> 항목을 추가하여라

3) 2번의 딕셔너리(dict)에서 Key 항목만 리스트로 저장하여 정렬한 후 튜플로 변경하여라.

결과 => ('성인', '소아', '아동', '청소년')



# 퀴즈

사원 정보에 대한 데이터를 딕셔너리 변수로 선언한 후 주석문 형태를 참조하여 데이터를 추가, 수정하는 코드를 작성하여라

```
employee_info = {
 "name": "김민수",
 "department": "AI 개발팀",
 "salary": 4500
}
```

# 퀴즈

=== 스타트업 인사 관리 시스템 ===

-----  
[초기 사원 정보 조회]

- 사원 이름 : 김민수
- 소속 부서 : AI 개발팀

-----  
[연봉 협상 및 인사평가 반영 결과]

- 변경된 연봉 : 5200 만원
- 올해의 등급 : A 등급

-----  
[최종 사원 디렉터리 데이터]

{'name': '김민수', 'department': 'AI 개발팀', 'salary': 5200, 'rating': 'A'}

# 퀴즈

- # 1. 딕셔너리 데이터 정의
- # 2. 데이터 초기 사원 정보 조회 및 출력 (사원이름, 소속 부서)
- # 3. 연봉 데이터 수정 및 새로운 항목 등급 추가
- # 4. 수정된 결과 출력

## 퀴즈

리스트안에 딕셔너리 형태로 컨퍼런스 참가자 데이터를 정의한 후  
주석문 형태를 참조하여 데이터를 조회하는 코드를 작성하여라

```
visitor_list = [
 {"name": "김철수", "company": "드림테크", "role": "개발자"},
 {"name": "이영희", "company": "에이아이랩", "role": "기획자"},
 {"name": "박민준", "company": "미래연구소", "role": "연구원"}
]
```

# 퀴즈

>> 출력 예시

[특정 데이터 콤 집어서 조회하기]

1. 1번째 참가자 전체 정보 : {'name': '김철수', 'company': '드림테크', 'role': '개발자'}
2. 2번째 참가자의 소속 회사 : 에이아이랩
3. 3번째 참가자의 담당 역할 : 연구원

# 퀴즈

>> 출력 예시

[종합 명단 서식 출력]

- 이름: 김철수 | 회사: 드림테크 | 역할: 개발자
- 이름: 이영희 | 회사: 에이아이랩 | 역할: 기획자
- 이름: 박민준 | 회사: 미래연구소 | 역할: 연구원

# 퀴즈

# 1) 0번 인덱스(첫 번째 사람)의 딕셔너리 전체 출력

# 2) 1번 인덱스(이영희)의 '소속 회사' 조회

# 3) 2번 인덱스(박민준)의 '역할' 조회

# 4) f-string 포매팅을 활용한 종합 명단 출력

# 집합 자료형

- 집합에 관련된 것들을 쉽게 처리하기 위해 만들어진 자료형  
집합 연산 가능(교집합, 합집합, 차집합, 여집합 기능)
- 중복을 허용하지 않는다.
- 순서가 없다(Unordered). 인덱싱, 슬라이싱 불가능
- 집합 자료형 생성  
set() 을 이용하여 객체 선언  
집합이름 = set( [ data1, data2... ])



# 집합 자료형

## ■ 집합 생성

```
s1 = {'a', 'b', 'c', 'd'}
```

```
s2 = set([1, 2, 3, 4, 2, 2, 100])
```

```
s3 = set('도레미파솔라시')
```

```
s4 = set(('야구', '농구', '농구', '배구'))
```

```
s5 = set({'a': 'apart', 'b': 'banana', 'c': 'cup'})
```

```
s6 = set()
```

```
print(s1, type(s1), len(s1))
```

```
print(s2, type(s2), len(s2))
```

```
print(s3, type(s3), len(s3))
```

```
print(s4, type(s4), len(s4))
```

```
print(s5, type(s5), len(s5))
```

```
print(s6, type(s6), len(s6))
```

```
빈튜플 (), 빈리스트 [], 빈딕셔너리 {}, 빈집합 set()
```

```
s7 = {}
```

```
print(s7, type(s7), len(s7))
```

# 집합 자료형

- 집합 요소 추가

```
add(데이타)
```

```
update([데이타1, 데이타2 ...])
```

```
s1 = {'db'}
```

```
print(f'set1 = {s1}')
```

```
s1.add('python')
```

```
print(f'set1 = {s1}')
```

```
s1.update(['pandas', 'numpy', 'seaborn'])
```

```
print(f'set1 = {s1}')
```

# 집합 자료형

- 집합 요소 삭제

`remove(데이터)`

`del 집합명`

```
s1 = {'python','pandas', 'numpy', 'seaborn'}
```

```
print(f'set1 = {s1}')
```

```
s1.remove('numpy')
```

```
print(f'set1 = {s1}')
```

```
del s1
```

```
print(f'set1 = {s1}')
```

## 퀴즈

슬램덩크 캐릭터 데이터를 이용하여 집합 자료형을 정의하고 CRUD를 이용한 파이썬 프로그래밍 코드를 작성하여라.

**shohoku\_set = {"강백호", "서태웅", "채치수", "송태섭", "강백호"}**

# 퀴즈

## 1. Create : 초기 집합 생성 완료

- 북산고교 명단 : {'채치수', '서태웅', '송태섭', '강백호'}
- 

## 2. Read : 집합 데이터 조회

- 현재 등록된 총 선수 인원수 : 4 명
- 

## 3. Update : 선수 추가 등록

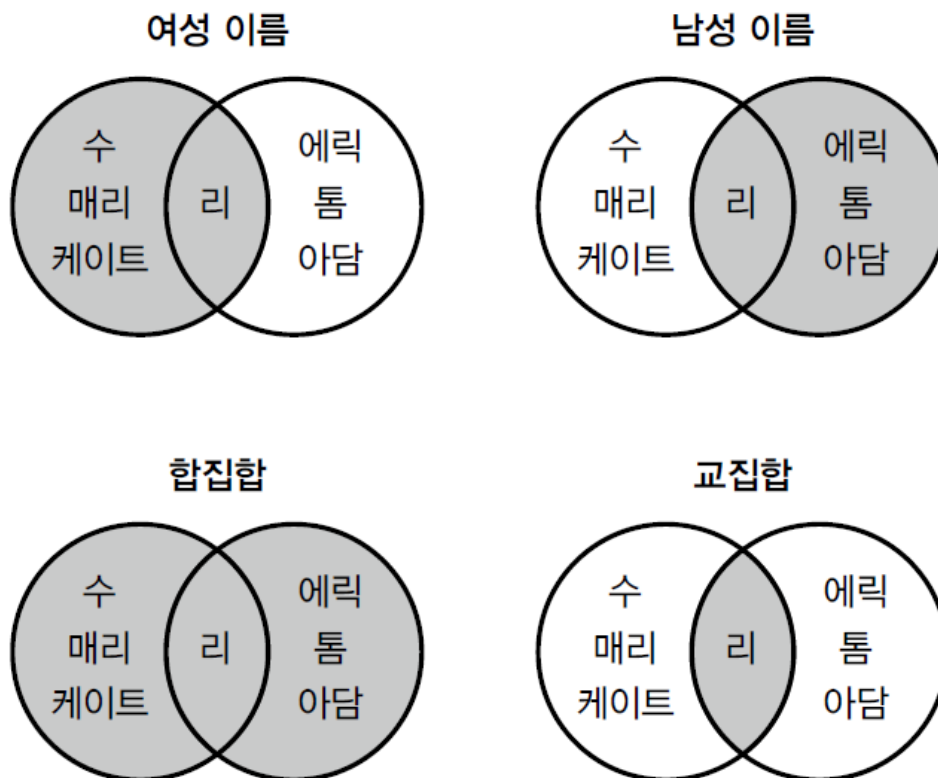
- 정대만 추가 후 명단 : {'채치수', '서태웅', '정대만', '송태섭', '강백호'}
  - 라이벌 추가 후 명단 : {'윤대협', '채치수', '서태웅', '정대만', '이정환', '송태섭', '강백호'}
- 

## 4. Delete : 명단에서 선수 제외

- 윤대협 제외 후 최종 명단 : {'채치수', '서태웅', '정대만', '이정환', '송태섭', '강백호'}

# 집합 자료형

- 집합 연산자와 집합 연산 함수



# 집합 자료형

- 합집합 1 - `setName1 | setName2`
- 합집합 2 - `setName1.union(setName2)`
  
- 차집합 1 - `setName1 - setName2`
- 차집합 2 - `setName1.difference(setName2)`
  
- 교집합 1 - `setName1 & setName2`
- 교집합 2 - `setName1.intersection(setName2)`
  
- 대칭차집합 1 - `setName1 ^ setName2`
- 대칭차집합 2 - `setName1.symmetric_difference(setName2)`

## 집합 자료형

```
set1 = {'최', '박', '선우', '김', '이'}
set2 = {'신', '장', '윤', '김', '이'}
print(f'{set1}|{set2} = {set1|set2}')print(f'set1.union(set2) = {set1.union(set2)}')
```

```
set1 = {'최', '박', '선우', '김', '이'}
set2 = {'신', '장', '윤', '김', '이'}
print(f'{set1} - {set2} = {set1-set2}')print(f'set1.difference(set2) = {set1.difference(set2)}')
```



# 집합 자료형

```
set1 = {'최', '박', '선우', '김', '이'}
set2 = {'신', '장', '윤', '김', '이'}
print(f'{set1} & {set2} = {set1&set2}')
```

```
print(f'set1.intersection(set2) = {set1.intersection(set2)}')
```

```
set1 = {'최', '박', '선우', '김', '이'}
set2 = {'신', '장', '윤', '김', '이'}
print(f'{set1} ^ {set2} = {set1^set2}')
```

```
print(f'set1.symmetric_difference(set2) = {set1.symmetric_difference(set2)}')
```

## 퀴즈

4개의 집합으로 정의된 OTT 서비스 데이터를 이용하여  
집합 연산을 적용하여 결과를 조회하는 파이썬 프로그래밍 코드를  
작성하여라.

```
netflix_shows = {"오징어게임", "더글로리", "지옥", "킹덤", "스위트홈"}
tvning_shows = {"더글로리", "삼시세끼", "지옥", "대탈출", "환승연애"}
disney_shows = {"무빙", "카지노", "지옥", "스타워즈", "어벤져스"}
```

```
친구가 꼭 보라고 강력 추천한 '인생작' 명단
recommended_shows = {"오징어게임", "더글로리", "무빙", "나의아저
씨", "미스터션샤인"}
```

# 퀴즈

>> 출력 예시

- 넷플릭스 콘텐츠 명단 : {'지옥', '더글로리', '오징어게임', '스위트홈', '킹덤'}
- 티빙 콘텐츠 명단 : {'지옥', '더글로리', '삼시세끼', '대탈출', '환승연애'}
- 디즈니플러스 명단 : {'지옥', '무빙', '카지노', '스타워즈', '어벤져스'}
- 친구의 강력 추천 명단 : {'무빙', '더글로리', '오징어게임', '나의아저씨', '미스터션샤인'}

# 퀴즈

[OTT 스마트 추천 시스템 연산 결과]

1. 가성비 최고 추천작 (추천 & 넷플릭스 & 티빙) : {'더글로리'}
2. 모든 플랫폼 공통 콘텐츠 (넷플릭스 & 티빙 & 디즈니) : {'지옥'}
3. 별도 개별 구매 필요 콘텐츠 (추천 - 나머지 전체) : {'나의아저씨', '미스터션샤인'}
4. 현재 넷플릭스나 티빙 중 내가 시청 가능한 전체 콘텐츠 목록 : {'지옥', '더글로리', '오징어게임', '스위트홈', '킹덤', '삼시세끼', '대탈출', '환승연애'}

## 퀴즈

number\_list에서 중복 숫자를 제거한 후 내림차순 형태로 정렬하여라

```
number_list = [5, 1, 2, 2, 3, 4, 5, 6, 7, 6, 7, 8, 9, 9, 10, 10]
```

예시)

Before : [5, 1, 2, 2, 3, 4, 5, 6, 7, 6, 7, 8, 9, 9, 10, 10]

After : [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

# 제 어 문

- 제어문이란?

프로그램의 흐름을 제어해서 효율적으로 이용하기 위한 명령문

- 제어문의 종류

- 조건문 : if / if ~ else / if ~elif~else

- 반복문 : for / while

- continue, break, pass

- 파이썬 제어문의 특징 :

- {} 사용하지 않고 탭1개 나 공백4칸 으로 블록 지정

- 들여쓰기가 없는 경우 IndentationError: 에러 발생

- switch 문이 없다

- else if 대신 elif 문이 있다

## ■ 들여쓰기(Indentation)

if 조건문:

수행할 문장

if 조건문:

수행할 문장1

수행할 문장2

수행할 문장3

들여쓰기 오류

X

## If 문

```
num = 20
print(num >= 20)
```

```
if (num >= 20):
 print(f'{num} 은 20보다 크다')
```

```
print('if 테스트 종료')
```

```
num = 20
```

```
if (num >= 20):
 print(f'{num} 은 20보다 크다') # IndentationError
```

```
print('if 테스트 종료')
```

# If 문

# if 안의 실행명령문이 하나인 경우에는 같은 행에 위치 가능

```
if (num >= 20) : print(f'{num} 은 20보다 크다')
```

```
myNum = int(input('숫자를 입력하여주세요...'))
```

```
if (myNum%2 == 0) : print('짝수이다.')
```

```
if (myNum%2 != 0) : print('홀수이다.')
```

```
print('if 테스트 종료')
```



# If 문

# 조건식대신에 변수나 값이 오는 형태

# True 조건이 되려면? 0, None, '' 값 외. 음수도 가능.

```
money = 0
if money:
 print('택시를')
 print(' 타고')
print('가자')
```

None, '' 가능

```
money = 2000
if money:
 print('택시를')
 print(' 타고')
print('가자')
```

# If ... else 문

- if .. else .. 문의 기본구조

if 조건문:

수행할 문장

...

else:

수행할 문장

...

```
myNum = int(input('숫자를 입력하여주세요...'))
```

```
if (myNum%2 == 0):
```

```
 print('짝수이다.')
```

```
else:
```

```
 print('홀수이다.')
```

# If .. elif 문

- 다중 조건 판단 elif 구조

```
if 조건문:
 수행할 문장
 수행할 문장
 ...
```

```
elif 조건문:
 수행할 문장
 수행할 문장
 ...
```

```
else:
 수행할 문장
 수행할 문장
 ...
```

```
num = int(input('num? ...'))
```

```
if num > 100:
 print(f' 입력 값 {num}은 100 보다 크다')
elif num < 100:
 print(f' 입력 값 {num}은 100 보다 작다')
else:
 print(f' 입력 값은 100이다')
```

```
print('if elif else 테스트 종료')
```

# 퀴즈

년도를 입력받은 후 윤년인지 평년인지 출력하도록 프로그래밍 하여라

# 윤년은 다음과 같은 규칙에 따라 결정된다.

- 연도가 4로 나누어 떨어지면 윤년.
- 그러나 연도가 100으로 나누어 떨어지면 윤년이 아니다.
- 다만 연도가 400으로 나누어 떨어지면 윤년.

연도를 입력하세요: 2000

2000년은 윤년입니다.

# 1900년 이후의 윤년

1904, 1908, 1912, 1916, 1920, 1924, 1928,  
1932, 1936, 1940, 1944, 1948, 1952, 1956,  
1960, 1964, 1968, 1972, 1976, 1980, 1984,  
1988, 1992, 1996, 2000, 2004, 2008, 2012,  
2016, 2020, 2024, 2028, 2032, 2036

# 퀴즈

입력 나이에 따른 메시지를 출력하여라

~7 : 영유아

8 ~ 13 : 초등학생

14 ~ 16 : 중학생

17 ~ 19 : 고등학생

20 ~ : 성인

당신의 나이를 입력해주세요? ...12

초등학생

## 퀴즈

키와 체중을 입력받은 후 bmi 값에 따라 메시지를 출력하여라.

# bmi 공식

k = 키(입력값) 단위 cm

w = 체중(입력값)

bmi = 체중(kg)/키(m)의제곱

키의 단위는 미터(m)

# bmi 값에 따라 출력되는 메시지

고도 비만 : 35 보다 클 경우

중등도 비만 : 30 - 35 미만

경도 비만 : 25 - 30 미만

과체중 : 23 - 25 미만

정상 : 18.5 - 23 미만

저체중 : 18.5 미만

키를 입력해주세요... 단위 cm...175

체중을 입력해주세요... 단위 kg...67

bmi = 21.88

정상

## 중첩 if 문

```
user_id = input('아이디 : ')
if user_id == 'python':
 user_password = input('패스워드 : ')
 if user_password == '1234': print('\n로그인 성공')
 else: print('\n비밀번호 오류')
else:
 print('\n아이디 오류')
```

## 퀴즈

고객이 입력한 나이와 키를 바탕으로, 올바른 자유이용권을 발급하고 특정 놀이기구의 탑승 가능 여부를 중첩 if 문을 이용하여 단계별로 판정하도록 프로그래밍 하여라

- 수강생 임의로 나이를 저장할 변수 age와 키를 저장할 변수 height를 선언하고 각각 값을 입력받는다

- 1차 조건 판정 (나이 기준):

나이가 만 12세 미만이면 '어린이'로 분류하고 만 12세 이상이면 '청소년 및 성인'으로 분류한다.



## 퀴즈

- 2차 중첩 조건 판정 (키 기준):

[어린이 분류인 경우]

키가 140cm 이상이면: "결과: [어린이 자유이용권] 발급 + 모든 놀이기구 탑승 가능!" 출력

키가 120cm 이상 140cm 미만이면: "결과: [어린이 자유이용권] 발급 + [보호자 동반] 시에만 탑승 가능합니다." 출력

키가 120cm 미만이면: "결과: [어린이 자유이용권] 발급 + 안전을 위해 유아 전용 기구만 이용 가능합니다." 출력

## 퀴즈

[청소년 및 성인 분류인 경우]

키가 150cm 이상이면: "결과: [일반 자유이용권] 발급 + 프리패스! 모든 기구 탑승 가능!" 출력

키가 150cm 미만이면: "결과: [일반 자유이용권] 발급 + 단, 일부 고공 스릴 기구는 탑승이 제한됩니다." 출력

나이가 만 12세 미만이면 '어린이'로 분류하고 만 12세 이상이면 '청소년 및 성인'으로 분류한다.

# 퀴즈

출력예시 >>

=== 스마트 놀이공원 입장 및 탑승 판정 시스템 ===

-----

만 나이를 입력하세요 (예: 11) : 12

키를 입력하세요 (cm 단위) : 135

-----

-> 기본 분류: 어린이 (만 12세)

결과: [어린이 자유이용권] 발급 + [보호자 동반] 시에만 탑승 가능합니다.

# If 문 – in List / not in List

- in / not in 연산자

데이터 in 튜플, 리스트, 문자열, 집합 => True / False

데이터 not in 튜플, 리스트, 문자열, 집합 => True / False

```
print('a' in 'banana')
```

```
print('가' in 'banana')
```

```
print('a' not in 'banana')
```

```
print('가' not in 'banana')
```

# If 문 – in List / not in List

if item in/not in group(리스트,튜플,문자열) :

명령문1

elif item in/not in group(리스트,튜플,문자열) :

명령문2

else:

명령문3

## If 문 – in List / not in List

```
blackpink = ['리사', '제니', '로제', '지수']
bts = ['진', 'RM', '지민', '뷔', '제이홉', '정국', '슈가']
member = input('member >> ')
if member in blackpink:
 print('블랙핑크')
elif member in bts:
 print('BTS')
else:
 print('블랙핑크도 BTS도 아니다')
```

## 퀴즈

If, in 연산자를 이용하여 다음과 같은 객관식 퀴즈를 프로그래밍 하여라.

다음 중 파이썬의 논리 연산자가 아닌 것은?

1. and
2. or
3. not
4. Plus

정답 번호를 입력하세요: 4  
정답입니다!

정답 번호를 입력하세요: 3  
틀렸습니다. 정답은 4번 'plus'입니다.